GRADUATION PROJECT

Team delivery plan

Workstream ownership from kickoff to delivery

A practical allocation for five members: main responsibilities, outputs in each phase, handoffs, and the GitHub task list. Each task has one owner; a second-person approval is not required.

Team
Abdelrahman Magdy
Abdullah Tarek
Abdelrahman Adel
Shahd Abdelhay
Dania El-Walily

Scope: Python; fine-tuning an existing model; no paid-services budget.

Schedule: an initial eight-week delivery window from the agreed kickoff. Official dates remain to be confirmed.

Status: foundation implemented; model training, final evaluation, and the complete product remain planned work.

Private GitHub repository and editable sources

Version 0.2. Read task status in GitHub Issues and the Project board.

Team delivery plan

Source: docs/planning/assignments.md

This plan assigns each member a main workstream from kickoff to the first delivery. Each task has one accountable owner. Larger outcomes are delivered through named handoffs between workstreams, not a separate peer-review process. The owner checks the work against the task's acceptance criteria and publishes evidence on GitHub.

Access at this edition: At the access check, Dania had Project write access but had not accepted the repository invitation. Her workstream and tasks are allocated in this plan; native Issue assignment will be completed after acceptance. This does not change her responsibilities.

Workstream ownership

Member	Main workstream	Main responsibility
Abdelrahman Magdy	Backend, integration, and coordination	API, run records, shared environment, connecting components, releases, and supervisor meetings
Abdullah Tarek	Agent workflow and verification	Analysis/refactoring/review-agent behavior, coordination, bounded retries, static checks, and the test runner
Abdelrahman Adel	Models and training	Model benchmarks, the local model provider, LoRA training, adapters, and inference settings
Shahd Abdelhay	Data, research, and evaluation	Related work, dataset preparation, leakage checks, experiment design, and final measurements
Dania El-Walily	Frontend and delivery materials	User interface, diff and result presentation, report assembly, slides, and demonstration materials

The review agent is a software component that checks model output. It is not a teammate assigned to review someone else's task. All members write the technical notes for their own work; Dania assembles the report and presentation rather than writing everyone else's contribution.

P1 Scope and feasibility - Week 1

Member	Work and output
Abdelrahman Magdy	Run kickoff; collect hardware and availability; help everyone reproduce the environment; confirm dates and report requirements; arrange the P1 supervisor meeting. Output: resource inventory, setup notes, and meeting decisions.
Abdullah Tarek	Define candidate refactoring types and examples. Work with Adel on feasibility and Shahd on source evidence. Output: a bounded task scope and before/after acceptance examples.
Abdelrahman Adel	Measure suitable models on available hardware and run a small LoRA smoke test. Output: model comparison, measured limits, and a saved/reloaded test adapter or a documented resource constraint.
Shahd Abdelhay	Read directly relevant studies and identify licensed data sources. Output: a focused reading table and a source/license shortlist.
Dania El-Walily	Map the input-to-result user journey and sketch the input, loading, diff, result, and failure screens. Output: a simple interface sketch for the scope discussion; no full frontend build is needed yet.

All five attend the supervisor discussion where available. Magdy records decisions and updates the plan. Scope approval follows the feasibility evidence; parallel sketches and candidate examples are preparatory work, not early approval of the scope.

P2 Data and baseline - Week 2

Member	Work and output
Abdelrahman Magdy	Specify API requests, responses, errors, and run states with Abdullah and Adel. Output: shared schemas and example JSON. Coordinate the P2 supervisor review.
Abdullah Tarek	Specify role inputs/outputs, tool boundaries, and retry limits. Output: a workflow design that matches the API and model-provider contracts.
Abdelrahman Adel	Implement the single-role baseline and fixed inference settings. Output: baseline results on development/validation examples and a reproducible command.
Shahd Abdelhay	Curate data, create source-separated splits, check leakage, and define the comparison protocol and targets after the baseline. Output: versioned data manifests and an agreed evaluation protocol.
Dania El-Walily	Finalize the screen layout against the agreed response schema, including missing tests and failed runs. Output: frontend interaction specification and reusable component plan.

Freeze the test set before model or prompt selection. Each member contributes their section of the supervisor evidence; Magdy schedules the meeting and records decisions.

P3 First integration and training - Weeks 3 and 4

Member	Work and output
Abdelrahman Magdy	Implement the run API and state records, connect components, and deliver one complete demonstration case. Output: an end-to-end run with clear errors and traceable settings.
Abdullah Tarek	Implement analysis and refactoring roles against the shared contracts. Output: structured findings and bounded proposals using Adel's model provider.
Abdelrahman Adel	Implement the local model provider and run the first fine-tuning experiment. Output: a reloadable adapter, configuration, and validation comparison.
Shahd Abdelhay	Maintain the data version used for training and prepare the evaluation harness. Output: case records, reference tests, and verified measurement inputs.
Dania El-Walily	Build the first frontend and connect it to the API. Output: code input, waiting/error states, explanation, and readable before/after diff.

The two main integration pairs are Adel + Abdullah for model-to-agent behavior and Magdy + Dania for API-to-interface behavior. Magdy connects these paths for the demonstration. P3 ends with a supervisor review of the working case and actual training results.

P4 Complete the first-release behavior - Weeks 5 and 6

Member	Work and output
Abdelrahman Magdy	Connect verification results to the API and run records, handle failure cases, and coordinate frontend integration with Dania. Output: a stable run contract and integrated result handling.
Abdullah Tarek	Complete role coordination, the review-agent step, bounded correction, and the separate runner for trusted examples. Execute the system acceptance checklist. Output: verified workflow behavior and acceptance evidence.
Abdelrahman Adel	Select the model using validation data and integrate the selected adapter or retain the baseline if justified. Output: model artifact, recorded selection evidence, and an updated model card.
Shahd Abdelhay	Dry-run the evaluation harness and verify case counts, denominators, timing, and failure records. Output: an evaluation process ready for the frozen test set.
Dania El-Walily	Complete result, missing-test, download, and failure presentation; begin the report from each member's technical notes. Output: a usable frontend and a report draft grounded in implemented work.

Freeze core features at the end of P4 after the supervisor discussion. Keep unresolved defects explicit rather than expanding the feature list.

P5 Evaluation and initial delivery - Weeks 7 and 8

Member	Work and output
Abdelrahman Magdy	Reproduce the system on a second machine, coordinate critical fixes, publish the release, and arrange the final supervisor review. Output: a tagged release, setup evidence, and delivery checklist.
Abdullah Tarek	Fix workflow and verification defects and document the implemented limits. Output: stable behavior and the workflow/testing sections for the report.
Abdelrahman Adel	Supply final model versions, inference settings, adapter checksums, and training records. Output: reproducible model artifacts and the training section.
Shahd Abdelhay	Run the independent A/B/C/D comparison and analyze failures and resource use. Output: complete results, figures, and the evaluation section.
Dania El-Walily	Assemble the report, slides, user guide, and demonstration recording from the team's real results. Output: final delivery materials. Each member presents their own work.

After the first delivery, retain the same workstream ownership and choose improvements from the supervisor feedback and evaluation findings. New dates and added features require a revised plan rather than an assumed commitment.

Handoffs

From	To	Required handoff
Shahd	Adel	Curated training/validation data, hashes, licenses, and split manifest
Adel	Abdullah	Model-provider interface, model/adapter revision, generation limits, and failure behavior
Abdullah	Magdy	Role schemas, workflow states, verification outputs, and execution limits
Magdy	Dania	API schemas, sample responses, and documented error states
All members	Dania	Technical notes, diagrams, actual results, and limitations for report assembly
All members	Magdy	Demonstration evidence and open decisions for each supervisor meeting

Simple working rules

1. Keep one primary implementation task active per person where practical. 2. Mark a task Ready only when its dependencies are satisfied; assignment alone does not start it. 3. Publish work through a linked pull request or a documented research output. A second-person approval is not required. The owner runs the relevant checks before merging or closing. 4. Keep task status and evidence in GitHub. Use the PDFs to understand the plan, not as a live status tracker. 5. Hold a supervisor review after every phase and record the decisions.

GitHub task allocation

The phase tables include ongoing work inside each member's workstream. The table below maps the tracked Issues to their accountable owners.

Task	Phase	Owner
T01 Run kickoff and record hardware and availability	P1	Abdelrahman Magdy
T02 Reproduce the environment on all team machines	P1	Abdelrahman Magdy
T03 Review related work and candidate data sources	P1	Shahd Abdelhay
T04 Benchmark candidate models on the demonstration machine	P1	Abdelrahman Adel
T05 Run a small LoRA feasibility experiment	P1	Abdelrahman Adel
T06 Agree on Python scope and supported refactoring types	P1	Abdullah Tarek
R1 Supervisor review: scope and feasibility	P1	Abdelrahman Magdy
T07 Curate a licensed sample dataset	P2	Shahd Abdelhay
T08 Split data and check leakage and duplicates	P2	Shahd Abdelhay
T09 Implement and measure the single-role baseline	P2	Abdelrahman Adel
T10 Freeze the comparison protocol and numerical targets	P2	Shahd Abdelhay
T11 Specify API and role contracts and run states	P2	Abdelrahman Magdy
R2 Supervisor review: data and baseline	P2	Abdelrahman Magdy
T12 Implement a replaceable local model provider	P3	Abdelrahman Adel
T13 Implement structured analysis-role outputs	P3	Abdullah Tarek
T14 Implement bounded refactoring proposals	P3	Abdullah Tarek
T15 Implement run API and state records	P3	Abdelrahman Magdy
T16 Build the initial code-input and result interface	P3	Dania El-Walily
T17 Run and record the first fine-tuning experiment	P3	Abdelrahman Adel
T18 Demonstrate an early end-to-end case	P3	Abdelrahman Magdy
R3 Supervisor review: integration and first training run	P3	Abdelrahman Magdy
T19 Implement review and role coordination	P4	Abdullah Tarek
T20 Design and verify a separate runner for trusted examples	P4	Abdullah Tarek
T21 Integrate verification and failure results into the interface	P4	Abdelrahman Magdy

Task	Phase	Owner
T22 Select and integrate the model using validation data	P4	Abdelrahman Adel
T23 Verify first-release acceptance criteria	P4	Abdullah Tarek
R4 Supervisor review: integrated first release	P4	Abdelrahman Magdy
T24 Run the independent final evaluation	P5	Shahd Abdelhay
T25 Verify second-machine setup and fix critical defects	P5	Abdelrahman Magdy
T26 Write the academic report and update the guides	P5	Dania El-Walily
T27 Prepare the presentation and repeatable demonstration	P5	Dania El-Walily
T28 Publish the initial delivery release	P5	Abdelrahman Magdy
R5 Supervisor review: initial delivery and next priorities	P5	Abdelrahman Magdy